

Recruit — Sistema ATS

Plataforma Integral de Gestión del Talento, Vacantes y Reclutamiento Asistido por IA

VERSIÓN	FECHA	AUTOR	DESTINATARIO
1.0	Julio 2026	Rodrigo Montero Duran	Equipo de Infraestructura / IT

1. Descripción General

Recruit es una aplicación web moderna de gestión del talento, vacantes y clientes (ATS - *Applicant Tracking System*). Centraliza el proceso de selección mediante la administración de candidatos, clientes, vacantes y la extracción automatizada de información de CVs utilizando Inteligencia Artificial (Azure OpenAI).

Usuarios Objetivo

Personal de reclutamiento, managers y administradores con cuentas validadas en Azure AD, con acceso controlado por dominios autorizados (whitelist) y roles explícitos en base de datos.

Módulos Principales

MÓDULO	DESCRIPCIÓN Y FUNCIONALIDAD
Candidatos	ABM de candidatos, parseo de CV con IA, gestión de etapas y perfiles de competencias.
Pipeline	Kanban interactivo para seguimiento visual y fluido del estado de los postulantes.
Vacantes	Búsquedas activas, estados, prioridades, rangos salariales y asignación directa a clientes.
Clientes	Gestión de empresas cliente y asociación en cascada con vacantes activas.
Seguridad	Configuración de dominios permitidos (whitelist), gestión centralizada de usuarios y asignación de roles.
Ajustes	Configuración de API Keys de IA y personalización de <i>branding</i> corporativo.

2. Stack Tecnológico

COMPONENTE	TECNOLOGÍA	VERSIÓN
Framework	Next.js (App Router)	16.2.6
Runtime	Node.js	≥ 20 LTS

COMPONENTE	TECNOLOGÍA	VERSIÓN
Lenguaje	TypeScript	5.x
ORM	Prisma	6.x
Base de Datos	PostgreSQL	15+
Autenticación	NextAuth.js + Azure AD (Entra ID)	v4.x
Estilos	Tailwind CSS	4.x
Package Manager	pnpm	9.x
IA & Parseo	OpenAI SDK / pdf-parse	6.x / 1.1.x

Dependencias de producción relevantes:

```
next, react, react-dom
@prisma/client, prisma
next-auth
openai, pdf-parse
@dnd-kit/core, @dnd-kit/sortable
lucide-react, recharts
```

3. Arquitectura de la Aplicación

La aplicación está diseñada bajo el modelo de un **monolito Next.js con App Router**. No depende de microservicios externos separados para la lógica de negocio.

```
Browser
├─ Next.js App (SSR + Client Components)
│   ├── Server Components → Leen DB directamente vía Prisma
│   ├── Server Actions → Mutaciones directas a DB sin API REST intermedia
│   ├── API Routes → /api/* (NextAuth y endpoint de parseo de CV)
│   └─ Client Components → UI reactiva (Kanban, modales), invoca Server Actions
```

Estructura de Directorios Relevante

```
src/
├─ app/
│   ├── (dashboard)/ # Rutas de negocio protegidas
│   ├── api/
│   │   ├── auth/[...nextauth]/ # Handler principal de NextAuth
│   │   └─ extract-cv/ # Endpoint POST para Azure OpenAI
│   ├── login/ # Pantalla de inicio de sesión
│   └─ unauthorized/ # Pantalla de acceso denegado
├─ components/ # Componentes React reutilizables
├─ lib/
│   ├── prisma.ts # Singleton de Prisma Client
│   └─ auth.ts # Utilidades y contexto de sesión
└─ prisma/
    └─ schema.prisma # Modelo de datos declarativo completo
```

Modo de renderizado: La aplicación utiliza *renderizado dinámico* en sus rutas de negocio para garantizar datos frescos en cada request y evitar problemas de caché con Next.js.

4. Base de Datos

Motor principal: **PostgreSQL 15+**. Es plenamente compatible con cualquier proveedor en la nube o hosting propio.

Entidades Principales del Modelo

TABLA	DESCRIPCIÓN Y PROPÓSITO
UserAccess	Whitelist de usuarios autorizados + asignación de rol (ADMIN , MANAGER , RECRUITER).
AllowedDomain	Whitelist de dominios corporativos permitidos para auto-registro/acceso.
Client	Empresas y Clientes de RMR.
JobOpening	Vacantes o búsquedas activas vinculadas a un cliente específico.
Candidate	Base global de talento (incluye posición, seniority, expectativa salarial, etc.).
Activity	Historial de interacciones, notas y cambios sobre candidatos.
AIConfig	Credenciales encriptadas de Azure OpenAI para análisis automático de CVs.
CompanyBranding	Configuración visual global (logo, colores primarios) persistida en base de datos.

Conexión y Migraciones: La aplicación utiliza conexión directa vía Prisma. La variable **DATABASE_URL** define el endpoint primario:

```
postgresql://USER:PASSWORD@HOST:PORT/DBNAME?schema=public
```

El proyecto utiliza el enfoque schema-driven mediante **npx prisma db push** para aplicar cambios al esquema sin archivos de migración formales.

5. Autenticación y Control de Acceso

Proveedor de Identidad: **Microsoft Azure AD (Entra ID)** mediante OAuth 2.0 / OIDC.

```
Usuario → /login
  → Redirect a Microsoft Login (OAuth)
  → Callback: NextAuth evalúa si la BD de usuarios está vacía
    └─ Si está VACÍA → Primer ingreso: Crea usuario como ADMIN y autoriza dominio (Bootstrapping)
    └─ Si NO está vacía → NextAuth verifica si el dominio está en AllowedDomain
      → NextAuth consulta tabla UserAccess en DB
      → Si el email está en whitelist → Sesión JWT creada
      → Si NO está → Redirect a /unauthorized
```

Control de Acceso en Dos Capas

- Dominio:** Solo cuentas cuyos dominios estén habilitados en AllowedDomain continúan el flujo.
- Whitelist:** Solo emails en la tabla UserAccess obtienen sesión. El rol ADMIN gestiona el acceso.

Requisitos de Configuración en Azure AD

Se requiere registrar un **App Registration** en el tenant corporativo con:

- **Redirect URI:** `https://[DOMINIO_PRODUCCION]/api/auth/callback/azure-ad`
- **Tipo de cuenta:** "Accounts in this organizational directory only" (Single Tenant).
- **Permisos OAuth:** `openid, profile, email`.
- **Consentimiento de Admin:** Otorgado para `User.Read`.

6. Variables de Entorno

Todas las variables deben definirse en el entorno de despliegue. **Ninguna credencial debe subirse al repositorio.**

VARIABLE	DESCRIPCIÓN	OBLIGATORIA
<code>DATABASE_URL</code>	Cadena de conexión completa a PostgreSQL.	SÍ
<code>NEXTAUTH_SECRET</code>	Secret para firmar tokens JWT (mínimo 32 caracteres aleatorios).	SÍ
<code>NEXTAUTH_URL</code>	URL pública canonical completa de la aplicación.	SÍ
<code>AZURE_AD_CLIENT_ID</code>	Client ID del App Registration en Azure AD.	SÍ
<code>AZURE_AD_CLIENT_SECRET</code>	Client Secret generado en Azure AD.	SÍ
<code>AZURE_AD_TENANT_ID</code>	Tenant ID del directorio corporativo de Azure AD.	SÍ

Comando para generar un secret seguro para `NEXTAUTH_SECRET`:

```
openssl rand -base64 32
```

7. Opciones de Despliegue e Infraestructura

Opciones de App Hosting

- **Opción A — Vercel (Recomendada):** Menor fricción operativa, soporte nativo para Serverless Functions y CI/CD integrado.
- **Opción B — Contenedor Docker (VPS / On-Premise):** Empleando output: `'standalone'` en la configuración de Next.js.
- **Opción C — Azure App Service / Container Apps:** Integración directa con el ecosistema Azure existente.

Opciones de Hosting de Base de Datos

- **Opción A — Supabase / Neon:** Base de datos PostgreSQL serverless de alto rendimiento.
- **Opción B — Azure Database for PostgreSQL:** Server Flexible con soporte SSL.
- **Opción C — AWS RDS / GCP Cloud SQL:** PostgreSQL gestionado estándar.

8. Ciclo de Vida: Build & Deploy

Comandos para el build de producción:

```
pnpm install
npx prisma generate
pnpm build
```

Pasos para el primer despliegue en un entorno nuevo:

```
# 1. Instalar dependencias
pnpm install

# 2. Empujar la estructura del esquema a la DB
npx prisma db push

# 3. Autoinicialización (El primer login web se asignará como ADMIN automáticamente)
```

9. Requisitos de Infraestructura — Resumen

RECURSO	MÍNIMO RECOMENDADO	NOTAS
Compute	512 MB RAM, 0.5 vCPU	Next.js standalone (Node 20) o Serverless.
Base de datos	PostgreSQL 15+ (SSL)	Crecimiento en función del volumen de candidatos.
Almacenamiento	Disco local / Blob	CVs guardados temporalmente en <code>/public/uploads</code> .
TLS / SSL	Obligatorio	NextAuth requiere HTTPS en entornos de producción.
Dominio	<code>recruit.empresa.com</code>	Coincidencia exacta con <code>NEXTAUTH_URL</code> y Azure AD.

10. Checklist de Migración

Azure AD

- ☐ Crear/Reutilizar App Registration en tenant corporativo.
- ☐ Configurar Redirect URI: `https://[DOMINIO]/api/auth/callback/azure-ad`
- ☐ Otorgar Admin Consent al permiso `User.Read`.
- ☐ Obtener Client ID, Client Secret y Tenant ID.

Base de Datos

- ☐ Crear instancia PostgreSQL 15+ en la infraestructura destino.
- ☐ Ejecutar `npx prisma db push`.
- ☐ Verificar conectividad desde el servidor de aplicaciones.

Despliegue & Validación

- ☐ Configurar las 6 variables de entorno requeridas.
- ☐ Configurar registros DNS, Dominio y Certificados TLS.
- ☐ Realizar primer login con el usuario ADMIN inicial.
- ☐ Navegar a *Seguridad* → *Configurar API de Azure OpenAI* y probar la extracción de CV.

11. Consideraciones de Seguridad

- Sin credenciales en repositorio: Gitignore estricto sobre `.env`.

- **Autenticación Obligatoria:** Todos los layouts y Server Actions requieren sesión activa.
- **Bootstrapping Seguro:** La asignación automática de ADMIN solo ocurre si UserAccess está totalmente vacía.
- **Aislamiento de Sesión:** Rotar NEXTAUTH_SECRET invalida todas las sesiones activas en caso de contingencia.

12. Contacto Técnico

ROL	PERSONA / REFERENTE	CONTACTO
Desarrollo / Owner	RODRIGO MONTERO DURAN	rmontero@rmrconsultores.com
Infraestructura / IT	Equipo de Infraestructura	—
Azure AD / IT Admin	Administrador Tenant Azure	—